

# 四電一丙 數位邏輯實習

## 第 8 次實習作業報告

組員 1 學號:B812111001 姓名: 張博翔

組員 2 學號: 114310326 黃子軒

組員 1 於此作業中的貢獻:解題、digital、Quartus、協助隊友排錯、作業二、作業報告(二次編排、修改與美編)

組員 2 於此作業中的貢獻:解題、digital、quartus、作業一、作業報告(初步編排、複審內容)

### I. 實驗目的

#### A. 作業 1

利用 J-K 正反器推導出下頁狀態圖的計數器電路，挑選以下的一組設計模式完成本題的計數器，第一組: 投影片 32 頁的設計方式 + 拉電路，第二組: 投影片 32 頁的設計方式 + 挑選投影片 30-31 頁的一種設計方式，狀態機的布林代數式推導過程必須詳細呈現於報告中。

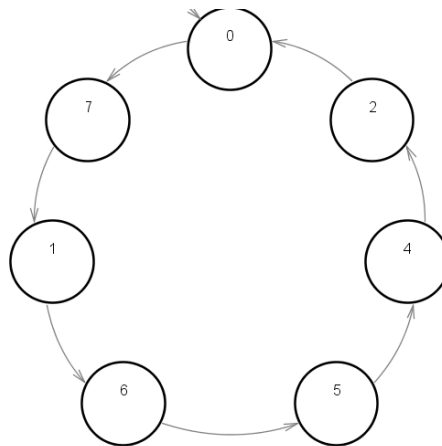


圖 1. J-K flip flop 計數器 state graph

#### B. 作業 2

利用 T 正反器推導出下頁狀態圖的狀態機電路，挑選以下的一組設計模式完成本題的狀態機，第一組: 投影片 32 頁的設計方式 + 拉電路，第二組: 投影片 32 頁的設計方式 + 挑選投影片 30-31 頁其中一種設計方式，布林代數式推導過程必須詳細呈現於報告中。

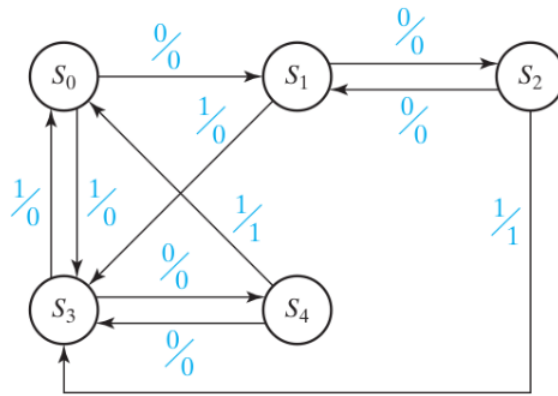


圖 2. Mealy state machine state graph

## II. 設計程序

### A. 作業 1

- 依照題目狀態  $000 \rightarrow 111 \rightarrow 001 \rightarrow 110 \rightarrow 101 \rightarrow 100 \rightarrow 010 \rightarrow 000$  建立狀態表，我們定義目前狀態為  $A B C$ ，次態為  $A^+ B^+ C^+$  並且根據 J-K 正反器的特性，來找出來找出輸入端 J 與 K 的值，接著卡諾圖進行化簡，最後根據卡諾圖化簡結果，最終得到布林代數的邏輯，以下進行推導布林代數式的過程，最後用 VHDL 程式碼撰寫狀態機。

表 1. State table

| $A B C$ | $A^+ B^+ C^+$ |
|---------|---------------|
| 000     | 111           |
| 111     | 001           |
| 001     | 110           |
| 110     | 101           |
| 101     | 100           |
| 100     | 010           |
| 010     | 000           |
| 011     | XXX           |

表 2. JK flip flop output

| $JK$ | $Q^+$ |
|------|-------|
| 00   | $Q$   |
| 01   | 0     |
| 10   | 1     |
| 11   | $Q'$  |

接著需要求出三個正反器 FF2、FF1、FF0 對應為  $J_c K_c$ 、 $J_b K_b$ 、 $J_a K_a$  的值，把表格列好，利用 JK 正反器來求輸入值，值填入卡諾圖進行化簡，最後求出布林代數式。

表 3. State Transition and Excitation Table

| 目前狀態 (CBA) | 下一個狀態 ( $C^+B^+A^+$ ) | $J_CK_C$ | $J_BK_B$ | $J_AK_A$ |
|------------|-----------------------|----------|----------|----------|
| 000        | 111                   | 1X       | 1X       | 1X       |
| 001        | 110                   | 1X       | 1X       | X1       |
| 010        | 000                   | 0X       | X1       | 0X       |
| 011        | XXX (未使用)             | XX       | XX       | XX       |
| 100        | 010                   | X1       | 1X       | 0X       |
| 101        | 100                   | X0       | 0X       | X1       |
| 110        | 101                   | X0       | X1       | 1X       |
| 111        | 001                   | X1       | X1       | X0       |

|   |   | BA |    |    |    |
|---|---|----|----|----|----|
|   |   | 00 | 01 | 11 | 10 |
| C | 0 | 1  | 1  | X  | 0  |
|   | 1 | X  | X  | X  | X  |

**Jc**

圖 3.  $Jc = B'$

|   |   | BA |    |    |    |
|---|---|----|----|----|----|
|   |   | 00 | 01 | 11 | 10 |
| C | 0 | X  | X  | X  | X  |
|   | 1 | 1  | 0  | 1  | 0  |

**Kc**

圖 4.  $Kc = B \oplus A$

|    |    | C |   |
|----|----|---|---|
|    |    | 0 | 1 |
| BA | 00 | 1 | 1 |
|    | 01 | 1 | 0 |
|    | 11 | X | X |
|    | 10 | X | X |

**Jb**

圖 5.  $Jb = C' + A'$

|    |    | C |   |
|----|----|---|---|
|    |    | 0 | 1 |
| BA | 00 | X | X |
|    | 01 | X | X |
|    | 11 | X | 1 |
|    | 10 | 1 | 1 |

**Kb**

圖 6.  $Kb = C + A'$

|   |   | BA |    |    |    |
|---|---|----|----|----|----|
|   |   | 00 | 01 | 11 | 10 |
| C | 0 | 1  | X  | X  | 0  |
|   | 1 | 0  | X  | X  | 1  |

**Ja**

圖 7 .  $J_a = C'B' + CB$

|   |   | BA |    |    |    |
|---|---|----|----|----|----|
|   |   | 00 | 01 | 11 | 10 |
| C | 0 | X  | 1  | X  | X  |
|   | 1 | X  | 1  | 0  | X  |

**Ka**

圖 8 .  $K_a = B'$

最後總結布林代數式，FF2 :  $J_c = B'$  ,  $K_c = B \oplus A$  , FF1  $J_b = C' + A'$  ,  $K_b = C + A'$  , FF0  $J_a = C'B' + CB$  ,  $K_a = B'$  , 以推導解果進行模擬驗證。

- 此外本題也要求需以 State 描述的方式完成計數器，以下為 VHDL 程式碼設計與講解。

```

-- 引用 IEEE 標準函式庫
LIBRARY ieee;
USE ieee.std_logic_1164.all;

-- 定義實體 (Entity)：定義外部接腳 (I/O)
entity arc is
    port (
        clk    : in  std_logic;          -- 時脈輸入
        a_out   : out std_logic;          -- 輸出位元 Q2 (MSB)
        b_out   : out std_logic;          -- 輸出位元 Q1
        c_out   : out std_logic;          -- 輸出位元 Q0 (LSB)
    );
end arc;

-- 定義架構 (Architecture)：描述電路的行為邏輯
architecture Behavioral of arc is
    -- 定義內部信號 z，用於存儲目前的計數狀態，初始值設為 "000"
    signal z : std_logic_vector(2 downto 0) := "000";
begin

    -- 並行賦值語句：將內部狀態 z 的各個位元映射到輸出接腳
    a_out <= z(2); -- 對應 A (Q2)
    b_out <= z(1); -- 對應 B (Q1)
    c_out <= z(0); -- 對應 C (Q0)

    -- 同步程序 (Process)：由 clk 信號觸發
    process(clk)
    begin
        -- 在時脈上升沿 (Rising Edge) 進行狀態跳轉
        if rising_edge(clk) then
            -- 根據目前的狀態 z，決定下一個狀態 (Next State Logic)
            if z = "000" then          -- 0 -> 7
                z <= "111";
            elsif z = "111" then        -- 7 -> 1
                z <= "001";
            elsif z = "001" then        -- 1 -> 6
                z <= "110";
            elsif z = "110" then        -- 6 -> 5
                z <= "101";
            elsif z = "101" then        -- 5 -> 4
                z <= "100";
            elsif z = "100" then        -- 4 -> 2
                z <= "010";
            elsif z = "010" then        -- 2 -> 0
                z <= "000";
            else                        -- 處理未使用狀態 (例如 011)，強制回到 000 (Reset)
                z <= "000";
            end if;
        end if;
    end process;

end Behavioral;

```

圖 9. J-K 正反器狀態圖計數器 VHDL Process

VHDL 程式碼講解：

當有輸入訊號 CLK，輸出訊號 A、B、C，宣告內部訊號 z 用來儲存目前狀態，設定初始值設為 000。輸入與輸出間關係，根據投影片第 35 頁要求的序列進行狀態轉換進行 ABC=000，111、001、110、101....在偵測時脈的上升邊緣觸，模擬實際電路中邏輯閘關係。

另外將所有非預期的狀態（011）導回了 000，以避免電路卡在死循環中。因此循環為：

$$000 > 111 > 001 > 110 > 101 > 100 > 010 > 000$$

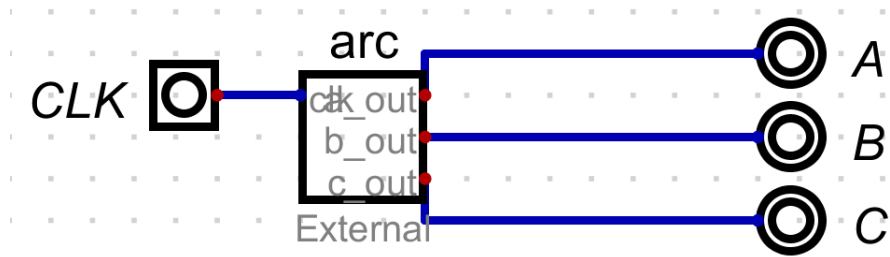


圖 10. J-K flip flop 計數器模擬電路 (解法 1)

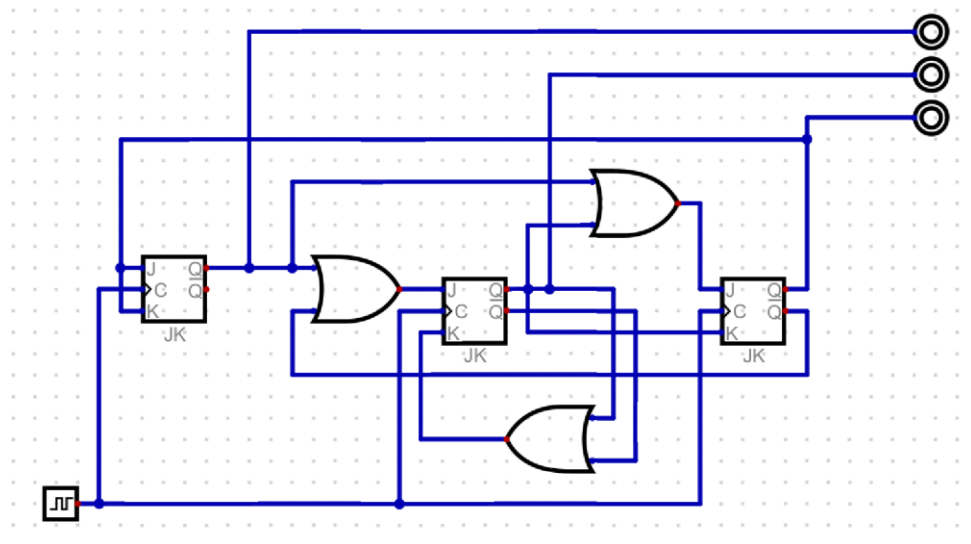


圖 11. J-K flip flop 計數器模擬電路 (解法 2)

## B. 作業 2

1. State Encoding，將 5 個狀態編碼以 3 位元二進位值表示 (Q2、Q1、Q0)。

- $S_0 = 000$
- $S_1 = 001$
- $S_2 = 010$
- $S_3 = 011$
- $S_4 = 100$
- (Don't Cares): 101, 110, 111

圖 12. State Encoding

2. 根據狀態圖推導出的狀態轉換與輸出如下表所示。其中次態與輸出取決於當前狀態以及輸入 (Mealy State Machine)。

| Present State | Next State ( $X = 0 \mid 1$ ) |            | Output $Z$ ( $X = 0 \mid 1$ ) |         |
|---------------|-------------------------------|------------|-------------------------------|---------|
| $Q_2Q_1Q_0$   | $X = 0$                       | $X = 1$    | $X = 0$                       | $X = 1$ |
| $S_0(000)$    | $S_1(001)$                    | $S_3(011)$ | 0                             | 0       |
| $S_1(001)$    | $S_2(010)$                    | $S_3(011)$ | 0                             | 0       |
| $S_2(010)$    | $S_1(001)$                    | $S_3(011)$ | 0                             | 1       |
| $S_3(011)$    | $S_4(100)$                    | $S_0(000)$ | 0                             | 0       |
| $S_4(100)$    | $S_3(011)$                    | $S_0(000)$ | 0                             | 1       |

圖 12. HW2 State table

3. 轉換為 Next state table 並進行卡諾圖化簡，將未使用狀態 (101, 110, 111) 視為 Don't Cares (d) 以求得最簡邏輯。

| $m$      | $Q_2$ | $Q_1$ | $Q_0$ | $x$ | $Q_2^+$ | $Q_1^+$ | $Q_0^+$ | $z$ | Note                  |
|----------|-------|-------|-------|-----|---------|---------|---------|-----|-----------------------|
| $m_0$    | 0     | 0     | 0     | 0   | 0       | 0       | 1       | 0   | $S_0 \rightarrow S_1$ |
| $m_1$    | 0     | 0     | 0     | 1   | 0       | 1       | 1       | 0   | $S_0 \rightarrow S_3$ |
| $m_2$    | 0     | 0     | 1     | 0   | 0       | 1       | 0       | 0   | $S_1 \rightarrow S_2$ |
| $m_3$    | 0     | 0     | 1     | 1   | 1       | 0       | 0       | 1   | $S_1 \rightarrow S_4$ |
| $m_4$    | 0     | 1     | 0     | 0   | 0       | 0       | 1       | 0   | $S_2 \rightarrow S_1$ |
| $m_5$    | 0     | 1     | 0     | 1   | 0       | 1       | 1       | 1   | $S_2 \rightarrow S_3$ |
| $m_6$    | 0     | 1     | 1     | 0   | 1       | 0       | 0       | 0   | $S_3 \rightarrow S_4$ |
| $m_7$    | 0     | 1     | 1     | 1   | 0       | 0       | 0       | 0   | $S_3 \rightarrow S_0$ |
| $m_8$    | 1     | 0     | 0     | 0   | 0       | 1       | 1       | 0   | $S_4 \rightarrow S_3$ |
| $m_9$    | 1     | 0     | 0     | 1   | 0       | 1       | 0       | 1   | $S_4 \rightarrow S_2$ |
| $m_{10}$ | 1     | 0     | 1     | 0   | $d$     | $d$     | $d$     | $d$ | invalid               |
| $m_{11}$ | 1     | 0     | 1     | 1   | $d$     | $d$     | $d$     | $d$ | invalid               |
| $m_{12}$ | 1     | 1     | 0     | 0   | $d$     | $d$     | $d$     | $d$ | invalid               |
| $m_{13}$ | 1     | 1     | 0     | 1   | $d$     | $d$     | $d$     | $d$ | invalid               |
| $m_{14}$ | 1     | 1     | 1     | 0   | $d$     | $d$     | $d$     | $d$ | invalid               |
| $m_{15}$ | 1     | 1     | 1     | 1   | $d$     | $d$     | $d$     | $d$ | invalid               |

圖 13. HW2 Next state table

| $Q_2x \setminus Q_1Q_0$ | 00                            | 01                                 | 11                                 | 10                                 |
|-------------------------|-------------------------------|------------------------------------|------------------------------------|------------------------------------|
| 00                      | 0 <sub><math>m_0</math></sub> | 0 <sub><math>m_2</math></sub>      | 1 <sub><math>m_6</math></sub>      | 0 <sub><math>m_4</math></sub>      |
| 01                      | 0 <sub><math>m_1</math></sub> | 1 <sub><math>m_3</math></sub>      | 0 <sub><math>m_7</math></sub>      | 0 <sub><math>m_5</math></sub>      |
| 11                      | 0 <sub><math>m_9</math></sub> | $d$ <sub><math>m_{11}</math></sub> | $d$ <sub><math>m_{15}</math></sub> | $d$ <sub><math>m_{13}</math></sub> |
| 10                      | 0 <sub><math>m_8</math></sub> | $d$ <sub><math>m_{10}</math></sub> | $d$ <sub><math>m_{14}</math></sub> | $d$ <sub><math>m_{12}</math></sub> |

圖 14. Q2+ K map    ( $Q2+ = Q_1'Q_0x + Q_1Q_0x'$ )

| $Q_2x \setminus Q_1Q_0$ | 00                 | 01                 | 11           | 10                 |
|-------------------------|--------------------|--------------------|--------------|--------------------|
| 00                      | $0_{m_0}$          | $\mathbf{1}_{m_2}$ | $0_{m_6}$    | $0_{m_4}$          |
| 01                      | $\mathbf{1}_{m_1}$ | $0_{m_3}$          | $0_{m_7}$    | $\mathbf{1}_{m_5}$ |
| 11                      | $\mathbf{1}_{m_9}$ | $d_{m_{11}}$       | $d_{m_{15}}$ | $d_{m_{13}}$       |
| 10                      | $\mathbf{1}_{m_8}$ | $d_{m_{10}}$       | $d_{m_{14}}$ | $d_{m_{12}}$       |

圖 15. Q1+ K map ( $Q1+ = Q_0'x + Q_2x' + Q_1'Q_0x'$ )

| $Q_2x \setminus Q_1Q_0$ | 00                 | 01           | 11           | 10                 |
|-------------------------|--------------------|--------------|--------------|--------------------|
| 00                      | $\mathbf{1}_{m_0}$ | $0_{m_2}$    | $0_{m_6}$    | $\mathbf{1}_{m_4}$ |
| 01                      | $\mathbf{1}_{m_1}$ | $0_{m_3}$    | $0_{m_7}$    | $\mathbf{1}_{m_5}$ |
| 11                      | $0_{m_9}$          | $d_{m_{11}}$ | $d_{m_{15}}$ | $d_{m_{13}}$       |
| 10                      | $\mathbf{1}_{m_8}$ | $d_{m_{10}}$ | $d_{m_{14}}$ | $d_{m_{12}}$       |

圖 16. Q0+ K map ( $Q0+ = Q_2'Q_0' + Q_0'x' = Q_0'(Q_2' + x')$ )

| $Q_2x \setminus Q_1Q_0$ | 00                 | 01                 | 11           | 10                 |
|-------------------------|--------------------|--------------------|--------------|--------------------|
| 00                      | $0_{m_0}$          | $0_{m_2}$          | $0_{m_6}$    | $0_{m_4}$          |
| 01                      | $0_{m_1}$          | $\mathbf{1}_{m_3}$ | $0_{m_7}$    | $\mathbf{1}_{m_5}$ |
| 11                      | $\mathbf{1}_{m_9}$ | $d_{m_{11}}$       | $d_{m_{15}}$ | $d_{m_{13}}$       |
| 10                      | $0_{m_8}$          | $d_{m_{10}}$       | $d_{m_{14}}$ | $d_{m_{12}}$       |

圖 17. Z K map ( $z = Q_1'Q_0x + Q_1Q_0'x + Q_2Q_0'x = x(Q_1'Q_0 + Q_0'Q_1 + Q_0'Q_2)$ )

推得布林代數式如下：

$$Q_2^+ = \overline{Q_1} Q_0 x + Q_1 Q_0 \overline{x} \quad (1)$$

$$Q_1^+ = \overline{Q_0} x + Q_2 \overline{x} + \overline{Q_1} Q_0 \overline{x} \quad (2)$$

$$Q_0^+ = \overline{Q_2} \overline{Q_0} + \overline{Q_0} \overline{x} = \overline{Q_0} (\overline{Q_2} + \overline{x}) \quad (3)$$

$$z = \overline{Q_1} Q_0 x + Q_1 \overline{Q_0} x + Q_2 \overline{Q_0} x = x [\overline{Q_1} Q_0 + \overline{Q_0} (Q_1 + Q_2)] \quad (4)$$

圖 18. HW2 Boolean Equations

4. 最後從以上 Boolean Equations 進行 VHDL 編寫。

```

library IEEE;
use IEEE.STD LOGIC 1164.ALL;

entity state machine is
    port (
        clk      : in  std logic;           -- 系統時脈輸入
        x        : in  std logic;           -- 外部輸入訊號 X
        z        : out std logic;           -- 米利型輸出訊號 Z
        q out    : out std logic vector(2 downto 0) -- 輸出當前狀態 Q2, Q1, Q0 (供觀察/除錯)
    );
end entity state machine;

architecture rtl of state machine is

    -- 內部狀態暫存器 (Current State Register)
    -- 給予初始值 "000" (代表開機後自動進入狀態 S0)
    signal Q : std logic vector(2 downto 0) := "000";

    -- 使用別名 (Alias) 將暫存器的位元命名，增加程式碼與布林式的一致性
    alias Q2 : std logic is Q(2);
    alias Q1 : std logic is Q(1);
    alias Q0 : std logic is Q(0);

    -- 次態連線 (Next State Wires)，用來存放計算後的下一個狀態數值
    signal Q2n, Q1n, Q0n : std logic;

begin

    -----
    -- 第一部份：組合邏輯 (Combinational Logic)
    -- 根據卡諾圖化簡 (考慮 Don't Cares) 得到的布林代數式
    -----

    -- 次態 Q2+ 的邏輯：當狀態為 S3 (011) 且 X=0 時，下個狀態為 S4 (100)
    -- 化簡式：Q2+ = Q1 * Q0 * X'
    Q2n <= Q1 and Q0 and (not x);

    -- 次態 Q1+ 的邏輯：由四個化簡後的乘積項組成
    -- 化簡式：Q1+ = Q2*Q0'*X' + Q1'*Q0*X' + Q2'*Q1'*X + Q1*Q0'*X
    Q1n <= (Q2 and not Q0 and not x) or
           (not Q1 and Q0 and not x) or
           (not Q2 and not Q1 and x) or
           (Q1 and not Q0 and x);

    -- 次態 Q0+ 的邏輯：由三個化簡後的乘積項組成
    -- 化簡式：Q0+ = Q0'*X' + Q2'*Q0' + Q2'*Q1'*X
    Q0n <= (not Q0 and not x) or
           (not Q2 and not Q0) or
           (not Q2 and not Q1 and x);

    -- 米利型輸出 Z 的邏輯：輸出取決於「當前狀態」與「目前輸入 X」
    -- 化簡式：Z = (Q1 + Q2) * Q0' * X
    z <= (Q1 or Q2) and (not Q0) and x;

    -----
    -- 第二部份：序向邏輯 (Sequential Logic)
    -- 負責在時脈上升沿將計算好的「次態」更新到「現態」暫存器中
    -----
    process(clk)
    begin
        if rising edge(clk) then
            -- 將三個位元組合 (Concatenation) 並存入狀態暫存器 Q
            Q <= Q2n & Q1n & Q0n;
        end if;
    end process;

    -----
    -- 第三部份：輸出賦值
    -- 將內部的暫存器數值拉到外部輸出埠 q out，以便在模擬波形中觀察狀態轉換
    -----
    q out <= Q;

end architecture rtl;

```

圖 19. HW2 VHDL Process (RTL)

5. 此外本題也要求需以 State 描述的方式完成計數器，以下為 VHDL 程式碼設計與講解。

```
library ieee;
use ieee.std_logic_1164.all;

entity mealy is
    port (
        -- Input ports
        clk, x : in std_logic;
        -- Output ports
        y       : out std_logic; -- 修正：補上分號
        z       : out std_logic_vector(2 downto 0)
    );
end mealy;

architecture state_graph of mealy is
    -- 必須宣告一個內部信號來存儲狀態，因為 out 腳位不能被讀取
    signal z_internal : std_logic_vector(2 downto 0) := "000";
begin
    -- 【新增】：將內部狀態信號即時輸出給腳位 z
    z <= z_internal;

    -- 以下的邏輯判斷，全部改用內部信號 z_internal
    y <= '1' when (z_internal = "010" and x = '1') or (z_internal = "100" and x = '1') else '0';

    process(clk) is
    begin
        if(rising_edge(clk)) then
            if(z_internal = "000") then -- 當前狀態 S0
                if(x = '0') then
                    z_internal <= "001"; -- x=0 跳至 S1
                else
                    z_internal <= "011"; -- x=1 跳至 S3
                end if;
            elsif(z_internal = "001") then -- 當前狀態 S1
                if(x = '0') then
                    z_internal <= "010"; -- x=0 跳至 S2
                else
                    z_internal <= "011"; -- x=1 跳至 S3
                end if;
            elsif(z_internal = "010") then -- 當前狀態 S2
                if(x = '0') then
                    z_internal <= "001"; -- x=0 跳至 S1
                else
                    z_internal <= "011"; -- x=1 跳至 S3
                end if;
            elsif(z_internal = "011") then -- 當前狀態 S3
                if(x = '0') then
                    z_internal <= "100"; -- x=0 跳至 S4
                else
                    z_internal <= "000"; -- x=1 跳至 S0
                end if;
            elsif(z_internal = "100") then -- 當前狀態 S4
                if(x = '0') then
                    z_internal <= "011"; -- x=0 跳至 S3
                else
                    z_internal <= "000"; -- x=1 跳至 S0
                end if;
            else
                z_internal <= "000"; -- 預防未知狀態，拉回 S0
            end if;
        end if;
    end process;
end state_graph;
```

圖 20. HW2 VHDL Process (Behavioral)

VHDL 程式碼講解：

實作 Mealy State Machine，其輸出訊號  $y$  的邏輯同時取決於目前的內部狀態與輸入  $x$ ，系統反應速度快。結束處使用了 `else` 語句處理未使用狀態（Don't Cares），確保系統若受到突發干擾進入非定義狀態（如 101, 110, 111）時，能自動回歸初始狀態  $S0$  進行防呆重置。

另外此次採用的是行為描述（Behavioral Description），利用高階的 `process` 行程與 `if-then-elsif` 條件分支來直接實作 State Diagram 的跳轉邏輯而非輸入布林代數式，相較起來更易於維護。

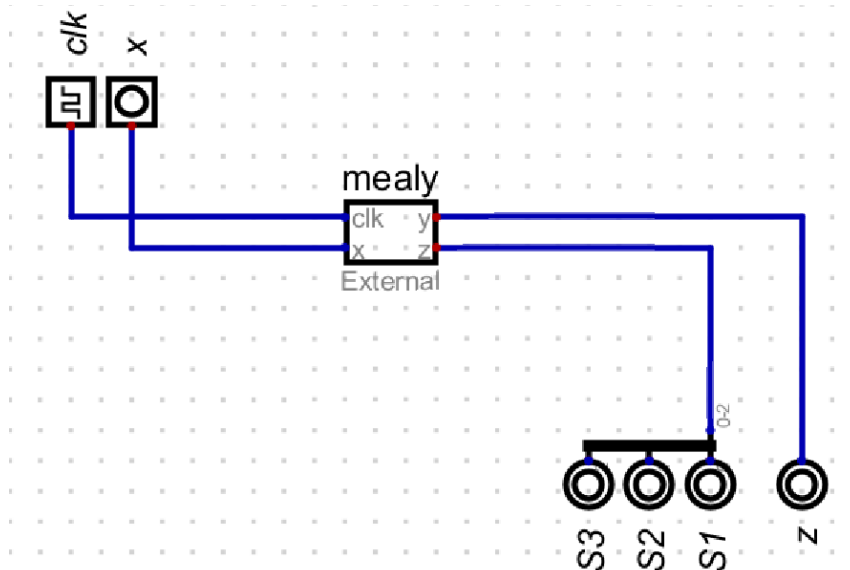


圖 21. HW2 VHDL 模擬電路 (RTL)

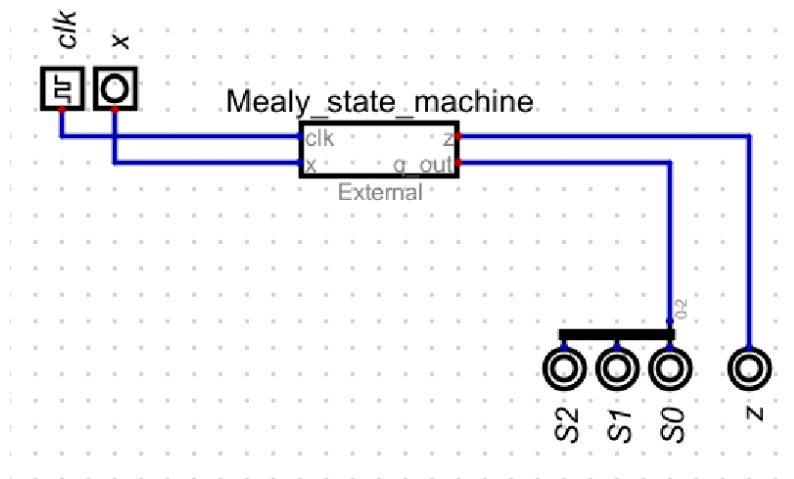


圖 22. HW2 VHDL 模擬電路 (Behavioral)

### III. 模擬驗證

#### A. 作業 1

模擬測試，測試並確認其功能是否是照著題目給的狀態圖進行，從 000 為初始點，開始接著與狀態圖計數器，在每一個時脈上升邊緣觸發後，輸出腳位 A、B、C 應依序出現以下數值：

(解法 1)：

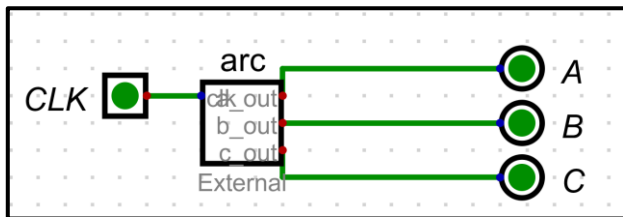


圖 23 . CLK 0，初始值為 000

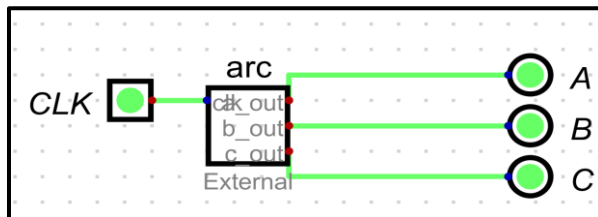


圖 24 . CLK 1，初始值為 111

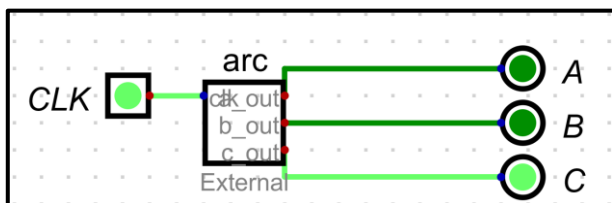


圖 25 . CLK 1，初始值為 100

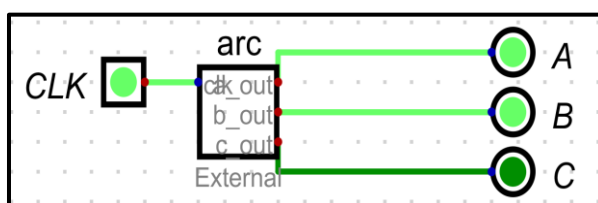


圖 26 . CLK 1，初始值為 011

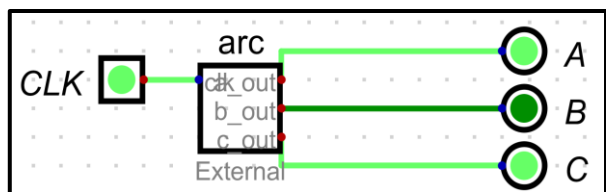


圖 27 . CLK 1，初始值為 101

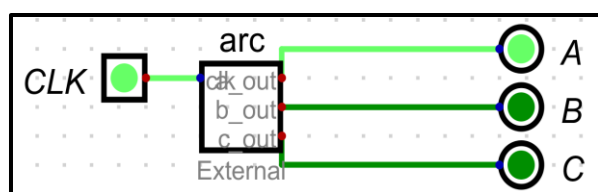


圖 28 . CLK 1，初始值為 001

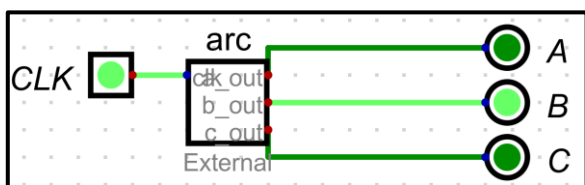


圖 29 . CLK 1，初始值為 101

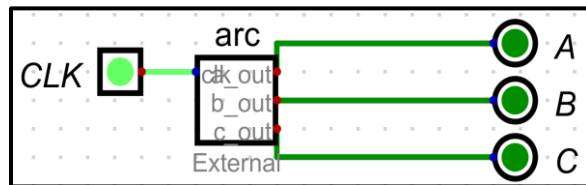


圖 30 . CLK 1，初始值為 000

(解法 2)：

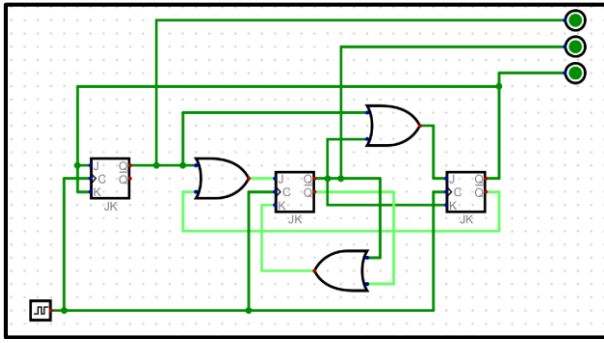


圖 31 . CLK 0 , 初始值為 000

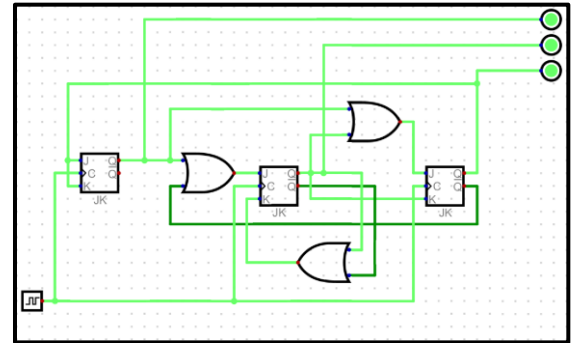


圖 32 . CLK 1 , 初始值為 111

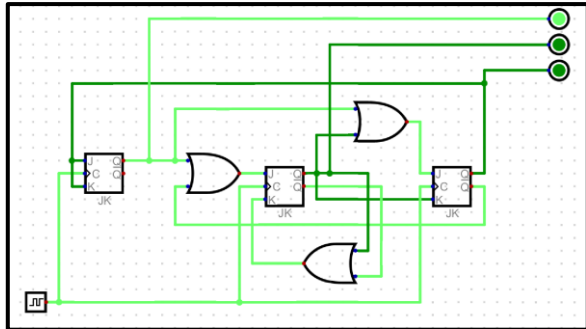


圖 33 . CLK 1 , 初始值為 100

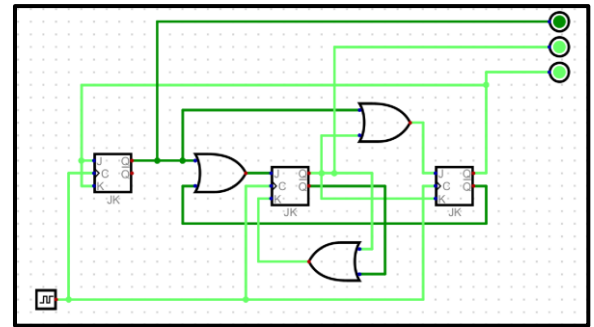


圖 34 . CLK 1 , 初始值為 011

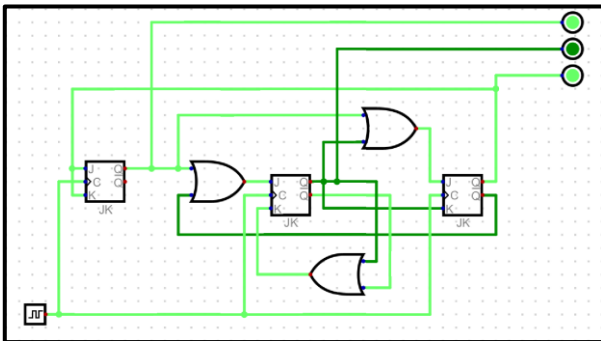


圖 35 . CLK 1 , 初始值為 101

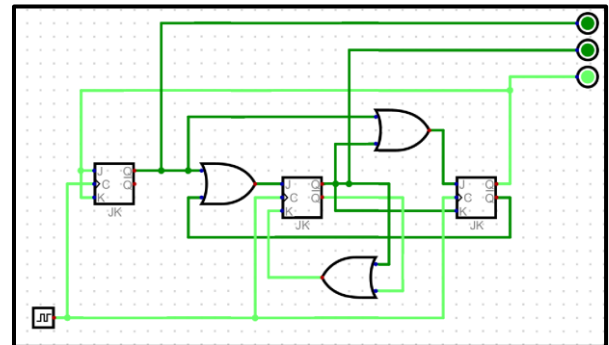


圖 36 . CLK 1 , 初始值為 001

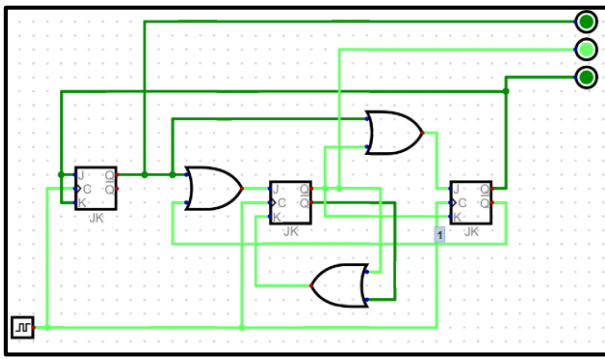


圖 37 . CLK 1 , 初始值為 101

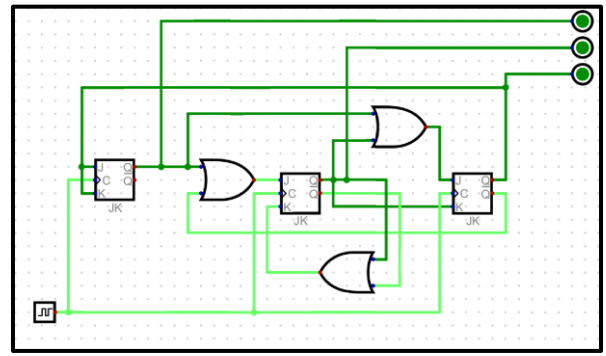


圖 38 . CLK 1 , 初始值為 000

## B. 作業 2

Mealy State Machine 實作 : input  $x$ 、 $clk$  ; output State( $S3 \sim S0$ )、 $z$

(解法 1) : RTL

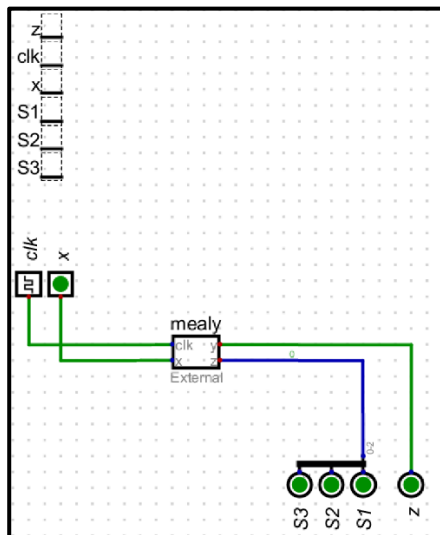


圖 39 .  $S0$  , Input 0 / Output 0 (進  $S1$ )

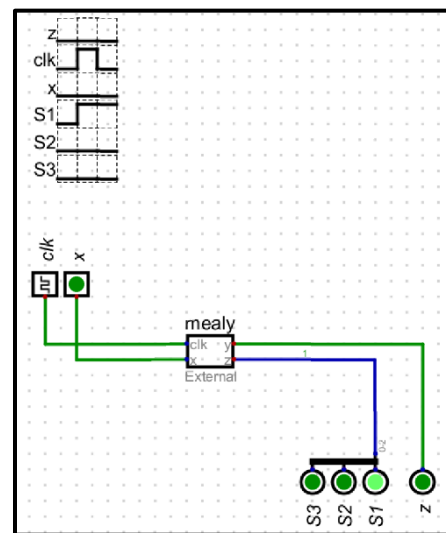


圖 40 .  $S1$  , Input 0 / Output 0 (進  $S2$ )

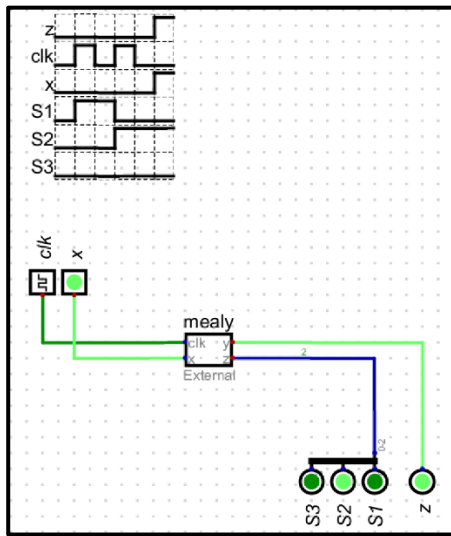


圖 41 . S2 , Input 1 / Output 1 (進 S3)

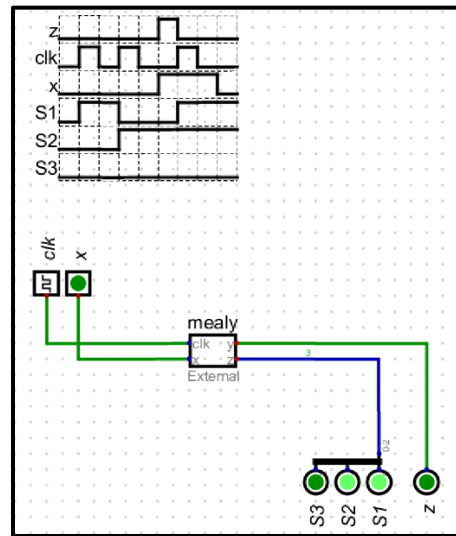


圖 42 . S3 , Input 0 / Output 0 (進 S4)

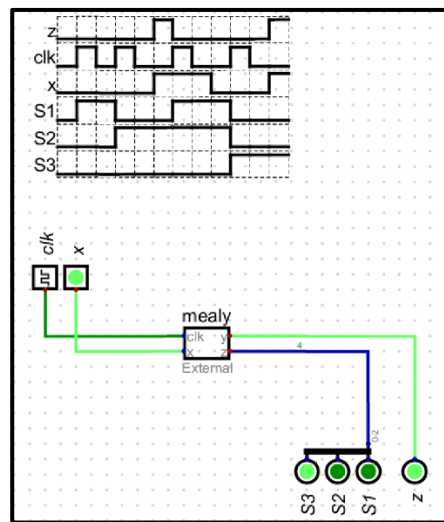


圖 43 . S4 , Input 1 / Output 1 (進 S0)

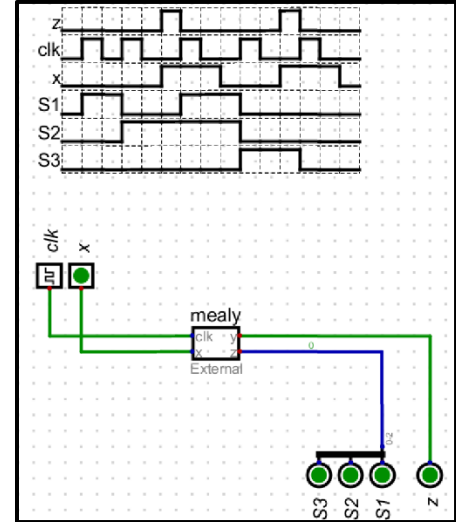


圖 44 . S0 , Input 0 / Output 0 (進 S1)

(解法 2) : Behavioral

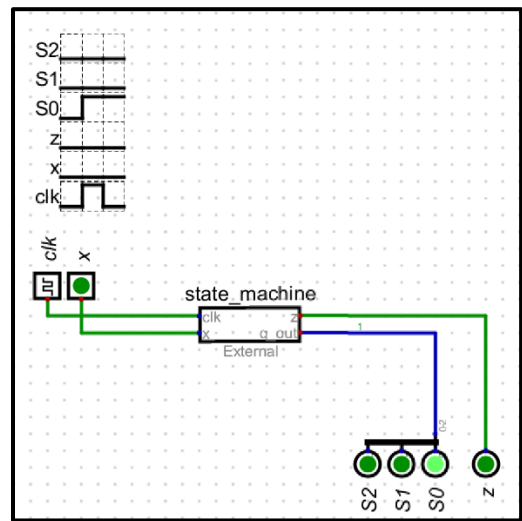
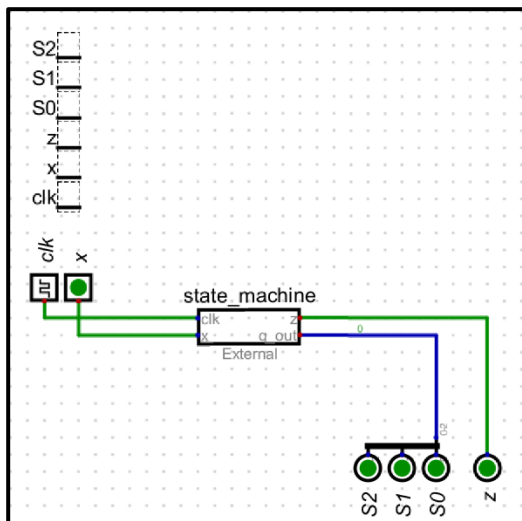


圖 45 . S0 , Input 0 / Output 0 (進 S1)

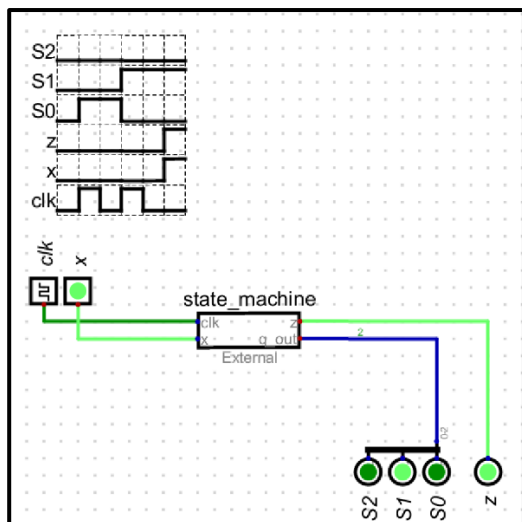


圖 47 . S2 , Input 1 / Output 1 (進 S3)

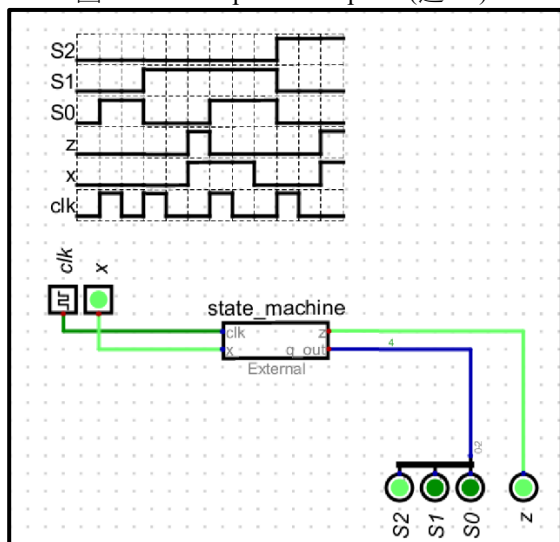


圖 49 . S4 , Input 1 / Output 1 (進 S0)

圖 46 . S1 , Input 0 / Output 0 (進 S2)

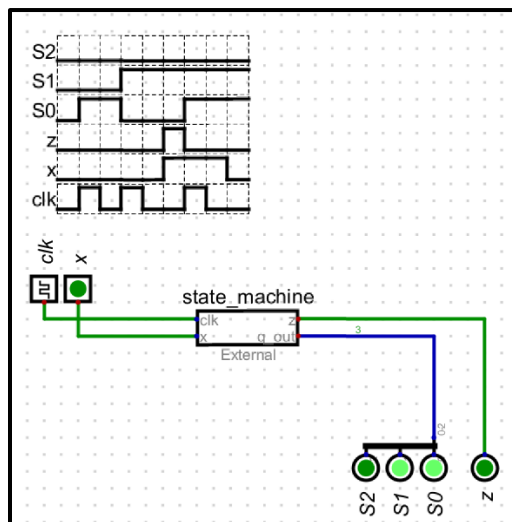


圖 48 . S3 , Input 0 / Output 0 (進 S4)

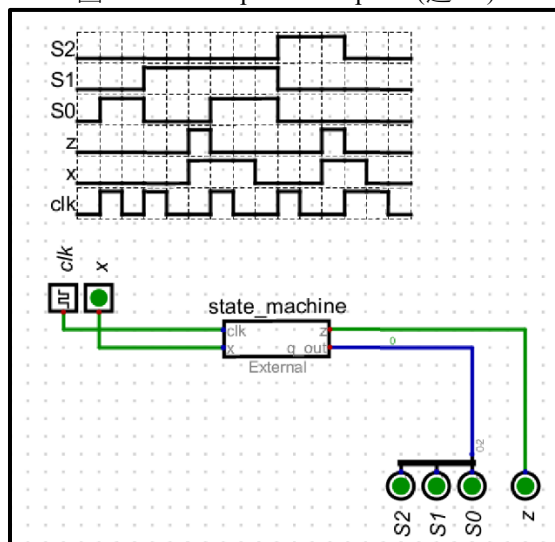


圖 50 . S0 , Input 0 / Output 0 (進 S1)

## 實驗結果與成果討論

### A. 作業 1

J-K 正反器狀態圖計數器：Input CLK；Output A、B、C；Output A、B、C 為 LED2、LED1、LED0，以下為實驗結果。

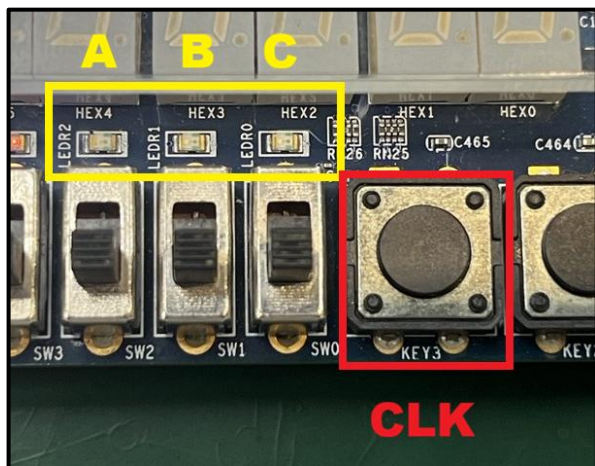


圖 51. CLK 0，輸出值為 000

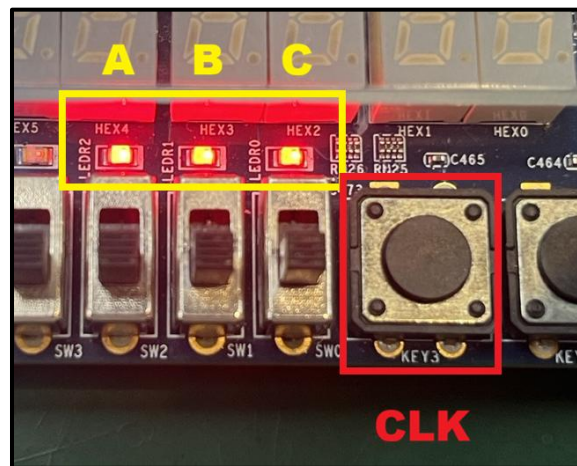


圖 52. CLK 1，輸出值為 111

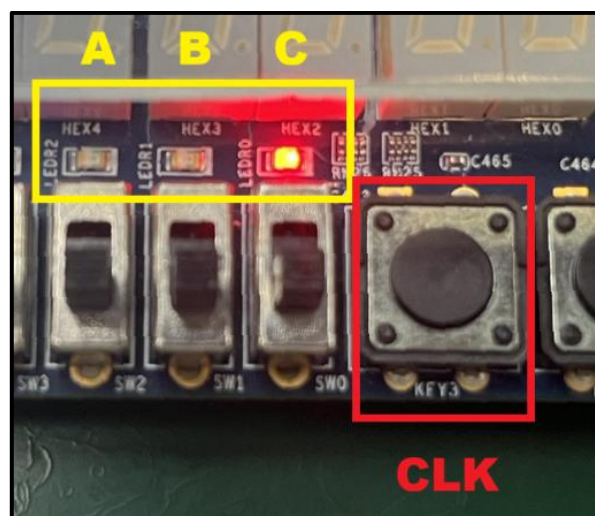


圖 53. CLK 1，輸出值為 001

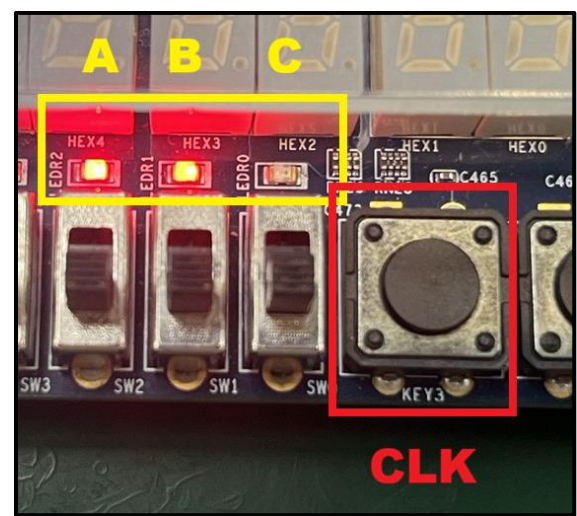


圖 54. CLK 1，輸出值為 110

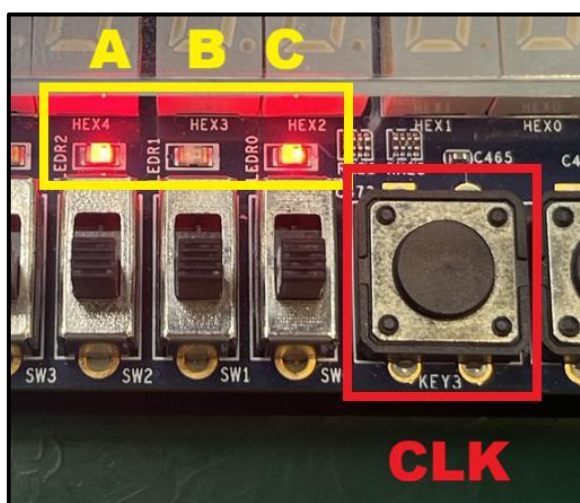


圖 55. CLK 1，輸出值為 101

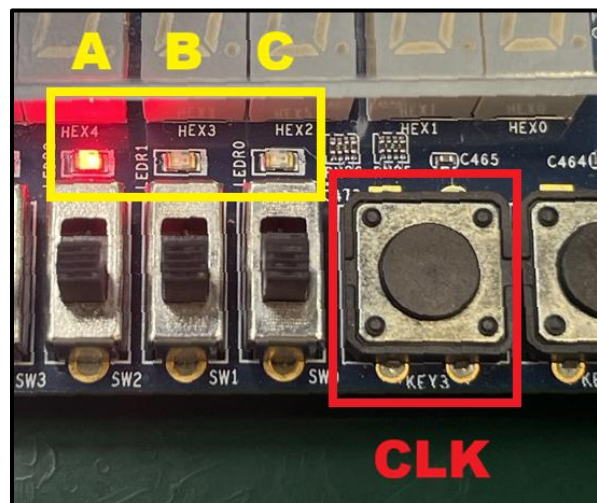


圖 56. CLK 1，輸出值為 100

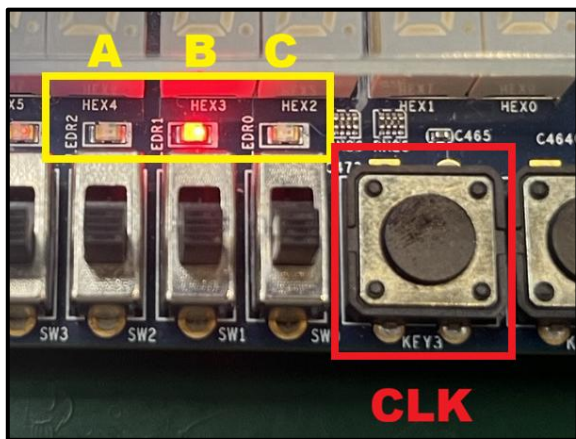


圖 57. CLK 1，輸出值為 010

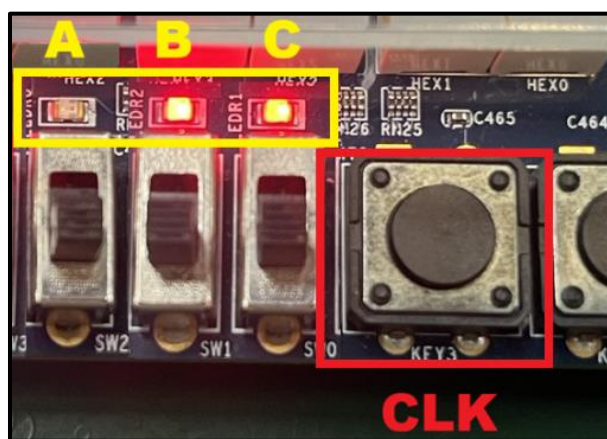


圖 58. CLK 1，輸出值為 011

## B. 作業 2

作業二無須燒入電路板實作，只需要模擬驗證。

以上實作兩種方式來控制當輸入超過記憶體容量(溢位)時的解決方式，就電路複雜度而言雖然比較偏好 disable register 的方式，但因為還有其他控制電路，因此最保險且最直觀的方式還是以控制寫入為主。

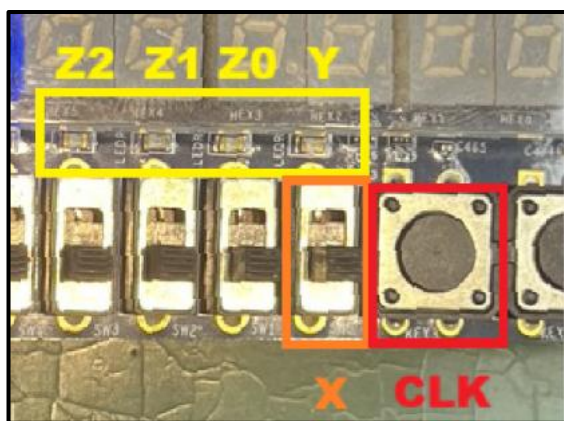


圖 59 . S0，Input 0 / Output 0 (進 S1)

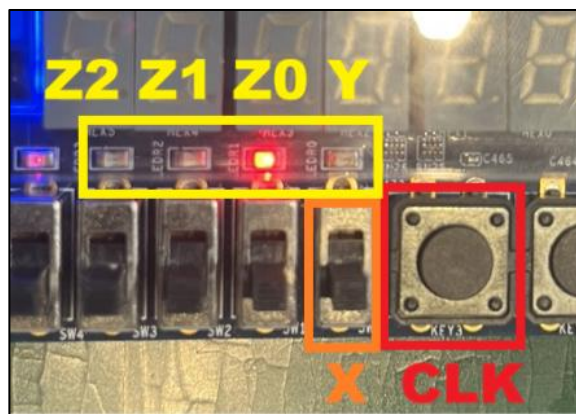


圖 60 . S1，Input 0 / Output 0 (進 S2)

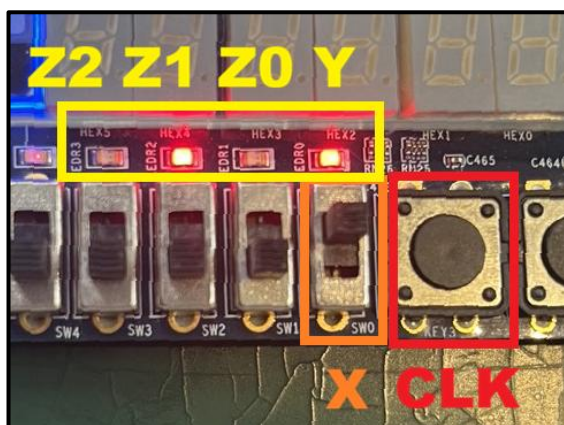


圖 61 . S2，Input 1 / Output 1 (進 S3)

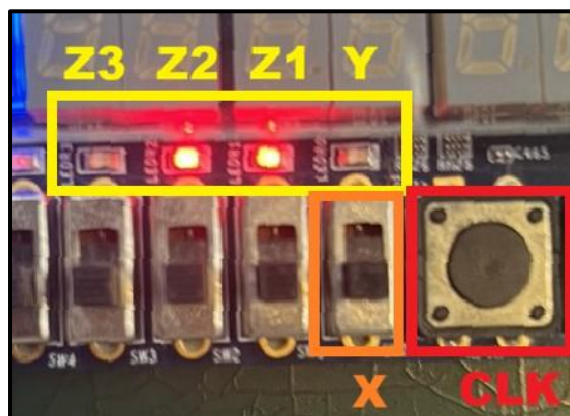


圖 62 . S3，Input 0 / Output 0 (進 S4)

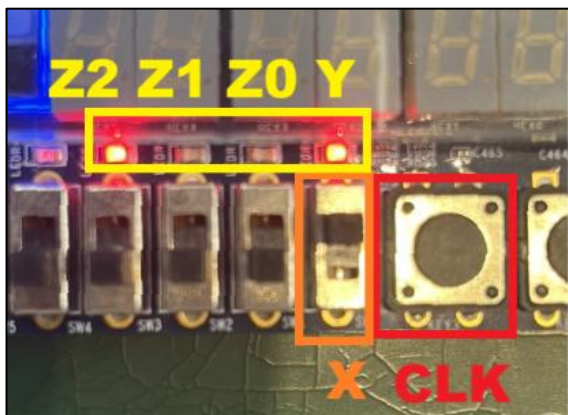


圖 63 . S4 , Input 1 / Output 1 (進 S0)

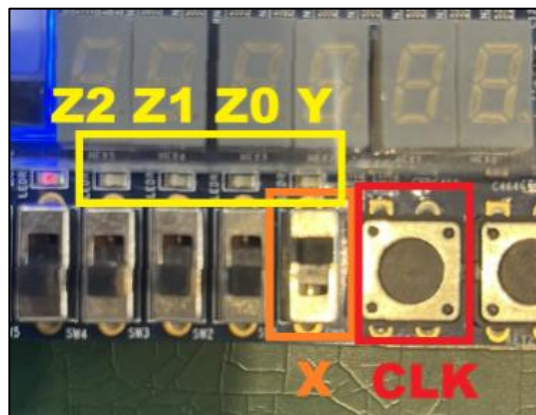


圖 64 . S0 , Input 0 / Output 0 (進 S1)

#### IV. 實驗心得

組員 1：開始與生活中的計算機元件連結，感覺非常有趣且更加容易做連結，希望可以有更多相關主題。

組員 2：這次作業一開始覺得有點難，尤其是時序的部分不太好理解。不過在實際畫波形後，才比較清楚。

#### 參考文獻

- [1] DLE 09 數位邏輯實習 計數器、狀態機